AQ-04 · attestqa

Low-Level Design

Version 1.4 (for sign-off) · 2026-10-10 · Author: Vijay Javvadi

1. Scope

Interfaces, data formats and module boundaries that implementation must follow. Where a section builds on code that already exists in agentic-qa, the file is named.

2. Core package (`packages/core`, npm `attestqa`)

2.1 What is carried over unchanged

The loop (`src/runtime/loop.ts`), model clients (`anthropic.ts`, `openai.ts`, `provider.ts`), ScopedGate and policy table, skills (`story-to-tests`, `story-context`, `story-rules`), adapters (`fs`, `pw`, `heal`, `summary`, tracker), triage (`src/triage/*`), rerun predicate (`src/runtime/rerun.ts`), scrubber, and the CLI. The refusal chain in `story-rules.ts` keeps its order and its run-id comments.

2.2 Ledger format and the 2.0 bump

Each line stays a `RunEvent` envelope: `{ tool, runId, eventId, timestamp, kind, payload }`. Today `tool` is the literal `"agentic-qa"` and `src/types.ts` declares that changing an exported shape is a major version.

Core therefore goes to **2.0.0** when desktop work starts (decision D7):

- `tool` becomes `"attestqa"`. Readers accept both values, so every 1.x ledger, including the Paper I corpus, still loads, replays and reports.
- Three event kinds are added:

Kind	Written by	Payload
`ruling`	UI action "rule: not a defect" / answering an open question	`{ subject: {type: "test"|"question"|"ac", ref}, decision, reason, author, source: "ui"|"tracker-comment" }`
`user_edit`	Specify "Edit text", or any write made through the UI	`{ path, before, after, author, reason? }`
`run_aborted`	Main process when a worker exits without `end`	`{ exitCode, signal?, lastEventId }`

- `approval_resolved` gains `author` (signed-in user or Windows account name). It is optional in the type so 1.x ledgers stay valid.
- The corpus stays on 1.x. Every new study records its own core commit (AQ-08).

2.3 Provider contract additions

`ModelClient` is unchanged. What changes is how a credential arrives. Today `buildModel()` reads a key from `RuntimeEnv`. 2.0 adds a token source:

```
export interface TokenSource {
  /** Called before every request. Must return a token valid for at least 60 s. */
  getToken(): Promise<{ token: string; expiresAt: number; header: "authorization" |
"api-key" }>;
  describe(): string;            // "Entra ID: vijay@acme.com" — shown in the policy
strip
}

export interface BuildModelOptions {
  config: ModelConfig;
  env: RuntimeEnv;
  promptVersion: string;
  modelOverride?: string;
  tokenSource?: TokenSource;      // new: wins over env when present
  warn?: (message: string) => void;
}
```

`ModelProvider` gains `"bedrock"` and `"gateway"`. `gateway` is OpenAI-compatible with arbitrary extra headers from config (never secrets; secrets come from the token source). `bedrock` is implemented in `@attestqa/provider-bedrock` against the Converse API and registered at start-up:

```
export function registerProvider(name: string, factory: (o: BuildModelOptions) =>
ModelClient): void;
```

Every provider must pass `test/provider-conformance.ts`, which generalises today's `test/provider.test.ts`: tool calls map back to qualified names; malformed arguments become `{}` with a warning; unknown tools are dropped with a warning; provider errors throw with scrubbed text; usage is recorded; no provider-side state is kept between `decide()` calls.

2.4 The explore skill

New skill `explore`, written to the same `Skill` interface as `story-to-tests`.

Tools offered (capability removal, not instruction): `browser.navigate`, `browser.snapshot`, `browser.click`, `browser.fill` (test-account values only, injected by the runtime, never visible to the model), `fs.read_file`, and `fs.write_file` limited by the gate to `features/drafts/**/*.feature`. Not offered: `pw.run_tests`, `tracker.*`, any write to `tests/` or `pages/`.

Refusal rules specific to explore:

Rule	Refuses
`verdictInExplore`	A draft containing pass/fail or defect language in a `Then` step's commentary, or any note claiming a bug

Rule	Refuses
`unobservedBehaviour`	A scenario step that cites no snapshot from this run (reuses the `unobservedClaim` pattern)
`outOfScopePage`	Navigation to a URL outside `scope.urls` or marked out of scope by the user

Output: drafts, a `sitemap.json` in `.aqa/runs/<id>/`, and open questions as `context` events with `{ question, page }`.

2.5 Config additions (`ai-quality.config.yaml`)

```
agent:
  mode: specify            # explore | specify | attest   (new)
  kind: api                # web | api | security | etl  (new; default web)
  model:
    provider: azure-openai
    name: gpt-4.1
    endpoint: https://acme.openai.azure.com
    auth: entra            # key | entra | aws-sso | none   (new; default key)
  scope:
    urls: ["http://localhost:3000"]
    api_hosts: ["localhost:3000"]          # (new) api: the only hosts requests may
reach
    work_items: ["44"]
  tracker:                                 # (new) optional
    type: ado                              # ado | jira
    project: JuiceShop-QA
    query: "Sprint 14"                     # or a saved-query id, or JQL for jira
    auth: entra                            # token from the vault or sign-in, never
here
  contracts:                               # (new) oracles beyond the story
    openapi: contracts/juice-shop.openapi.yaml
    mapping: contracts/sales-load.mapping.yaml
  security:                                # (new, v4.0)
    asvs: ["V3.3.1", "V4.1.1", "V4.2.1"]   # requirements the company chose
    active_scan: false                     # true only with authorisation in
targets.yaml
  etl:                                     # (new, v5.0)
    source: { driver: sqlite, path: ../data/ecommerce.db }
    target: { driver: sqlite, path: ../data/warehouse.db }
  approvals:
    timeout_minutes: 30    # (new) nothing is approved by default
  explore:
    max_pages: 40          # (new)
    out_of_scope: ["/#/score-board"]
```

`auth` names how a token is obtained; it never holds a secret. Database paths and hosts are not secrets; database passwords come from the vault like model keys. Config validation keeps today's behaviour: an unknown key or provider is an error that names the allowed values.

2.6 Test types: adapters and refusal rules

`kind` selects the adapter set and adds type-specific rules to the refusal chain. The existing chain runs for every type first; the type's rules run after it. Each rule follows the pattern of today's rules: it reads the proposed write and the ledger, returns a message that says how to clear it, and gets a test named after the run that first needed it.

Type	Adapter tools offered	Rule	Refuses a write that…
api	`api.request` (method, path, headers, body; host must be in `api_hosts`), `api.contract` (reads the OpenAPI operation), plus `fs`, `pw.run_tests`	`statusWidened`	replaces an exact expected status with a range, a list containing the observed status, or removes the status check, while the test is failing
		`schemaCheckDropped`	removes a call that validates the response against the contract
		`expectedFieldRemoved`	removes an expected field or value from a body assertion while the test is failing
security	`pw` passive checks; `zap.passive`, `zap.spider`; `zap.active` only if `targets.yaml` records authorisation for this origin	`findingSuppressed`	adds an ignore, filter or `test.skip` around a finding
		`allowListGrown`	adds an entry to an allow list or ZAP alert filter during a run that has an open finding
		`severityLowered`	lowers a severity or risk threshold that a failing check relies on
etl	`sql.query` (read-only), `sql.profile` (row counts, nulls, distinct, min/max per column), `contract.mapping`	`toleranceWidened`	raises a numeric tolerance in a reconciliation assertion while it is failing
		`rowFilterAdded`	adds a WHERE/filter clause to the source or target side of a failing reconciliation that the mapping does not specify
		`expectedSetToActual`	replaces an expected value with the value just observed in the ledger

Read-only SQL. `sql.query` parses each statement and accepts only `SELECT` and `WITH … SELECT`, with no `INTO` and no function the driver marks as side-effecting; anything else is refused before it reaches the database. Connections are opened read-only where the driver allows (SQLite `mode=ro`, PostgreSQL `default_transaction_read_only`). Both behaviours have their own tests.

Security authorisation. `zap.active` is in the `destructive` policy class. It is only added to the tool list when `targets.yaml` has `active_scan_authorised: {by, date, scope}` for the run's origin. Without that record the tool does not exist for the run, so no prompt can reach it.

Explore per type. API explore reads the OpenAPI file or a HAR captured during web explore and drafts request scenarios; security explore lists routes, forms, cookies and response headers; ETL explore runs `sql.profile` on both sides and drafts reconciliation checks from the mapping. All write drafts only, under the explore rules in §2.4.

2.7 Tracker interface

```
export interface TrackerAdapter {
  readonly kind: "ado" | "jira";
  listWork(q: WorkQuery): Promise<WorkItemRef[]>;          // sprint, area, saved
query, JQL
  getStory(id: string): Promise<Story>;                    // title, description,
acceptance criteria
  getComments(id: string): Promise<Comment[]>;             // rulings are comments with
an author
  getLinkedBugs(id: string): Promise<BugRef[]>;            // id, state, title, created
by agent?
  createBug(b: ConfirmedBug): Promise<BugRef>;             // only path that writes;
needs approval
}
```

`createBug` takes a `ConfirmedBug`, which can only be built from an `approval_resolved` event with `approve: true`, so the type system and the gate both stand between the agent and a tracker write. There is no update method. Jira acceptance criteria are read from a configurable field (custom field id or a description section), because Jira has no standard criteria field; ADO uses `Microsoft.VSTS.Common.AcceptanceCriteria`, as today.

2.8 Coverage

Generated tests carry tags such as `@story:44 @ac:4 @kind:api`. Coverage for the Requirements screen is the join of those tags (read from the code), approvals (from the ledger) and the latest result per test (from the latest full run's ledger). It is computed in main or the server, never by the model, and the same function backs the CLI command `aqa coverage`.

2.9 Dependencies on our packages

Package	Version range	Adapter / tool name	Notes
`@vijaypjavvadi/bdd2pw`	`^4.4`	`bdd2pw.scaffold` (exists)	Unchanged
`@vijaypjavvadi/pw-emit`	`^1.3`, then `^1.4`	used inside `specify`	1.4 adds IR nodes `ApiRequestTest`, `SecurityCheck`, `Reconciliation`; additive, so a minor release

Package	Version range	Adapter / tool name	Notes
`@vijaypjavvadi/synthdata`	current major	`synthdata.generate`	Replaces today's `SYNTHDATA_CMD` shell-out with the package API; the command form stays as a fallback
`@vijaypjavvadi/pw-self-heal`	`^1.2`	`heal.candidates`	Called with `mode: "suggest"`; the fixture that heals at run time is never installed in a governed project. A test checks that generated projects do not import it
`@vijaypjavvadi/sel2pw`	`^2.0`	`import.selenium` (first run only)	Runs once per import; not offered to the agent during ordinary runs
`@vijaypjavvadi/usagecontract`	`^1.6`	`api.contract_from_traffic` (v2.1)	Input is the HAR recorded in Explore; output goes to `contracts/` as a proposed write needing approval

All six are optional peers of core except `bdd2pw` and `pw-emit`, which core needs for Specify. A missing optional package removes its tool from the run's tool list, and the policy strip says so.

3. Client package (`packages/client`)

The UI's only dependency on the rest of the system:

```
export interface AttestClient {
  // projects and settings
  listProjects(): Promise<ProjectSummary[]>;
  openProject(path: string): Promise<Project>;
  testProvider(cfg: ModelConfig): Promise<{ ok: boolean; latencyMs: number; error?:
string }>;

  // runs
  startRun(req: { project: string; mode: Mode; story?: StoryRef; url?: string }):
Promise<{ runId: string }>;
  stopRun(runId: string): Promise<void>;
  events(runId: string, fromEventId?: number): AsyncIterable<AnyRunEvent>;   // live
tail
  report(runId: string): Promise<RunReport>;                                 // counted
from ledger, split by kind

  // requirements
  listWork(project: string, q: WorkQuery): Promise<WorkItemRef[]>;
  coverage(project: string, q: WorkQuery): Promise<CoverageGrid>;            // §2.8

  // human decisions
  answerApproval(runId: string, approvalId: number, a: { approve: boolean; reason?:
string }): Promise<void>;
  rule(runId: string, r: RulingInput): Promise<void>;
  editScenario(project: string, e: ScenarioEdit): Promise<void>;

  // ledger questions — answered by query, never by the model
  ask(runId: string | "all", q: LedgerQuery): Promise<LedgerAnswer>;
}
```

`IpcClient` implements it over the preload bridge; `HttpClient` implements it over REST and server-sent events. The UI is tested against a third implementation, `FakeClient`, that replays a recorded ledger.

`ask()` takes a structured query (`whyQuarantined(test)`, `refusalsFor(path)`, `runsTouching(route)`, `approvalsBy(user)`) parsed from the question box by a small grammar. A question the grammar cannot parse gets "I can't answer that from the ledger" and a list of questions it can. It never falls through to the model.

4. Winapp (Electron)

4.1 Process and file layout

```
Winapp/
  src/main/        index.ts, windows.ts, vault.ts, workers.ts, ipc.ts, updater.ts
  src/preload/     index.ts      (contextBridge.exposeInMainWorld("attest", api))
  src/worker/      run.ts        (utilityProcess entry; imports attestqa core)
  electron-builder.yml
```

`BrowserWindow` options: `contextIsolation: true`, `sandbox: true`, `nodeIntegration: false`, a strict Content-Security-Policy, navigation to any non-app URL blocked, `window.open` denied.

4.2 IPC contract

Every channel is typed in `packages/client/src/ipc.ts` and validated in main with a schema before use.

Channel	Direction	Payload	Notes
`run:start`	renderer → main	`StartRunRequest`	Main resolves credentials, spawns worker
`run:stop`	renderer → main	`{ runId }`	Worker gets a stop message; ledger gets `end`
`run:event`	main → renderer	`AnyRunEvent`	Forwarded from worker as written
`approval:request`	worker → main → renderer	`{ runId, approvalId, call, gate, evidence }`	Brings window to front
`approval:answer`	renderer → main → worker	`{ runId, approvalId, approve, reason?, author }`	`author` is added by main, not trusted from renderer
`vault:set`	renderer → main	`{ provider, secret }`	Write-only from the renderer's side; there is no `vault:get`
`provider:test`	renderer → main	`ModelConfig`	Real tool-call round trip
`ledger:ask`	renderer → main	`{ runId, query }`	Executed in main against the JSONL

4.3 Worker protocol

Main ↔ worker messages over the `utilityProcess` port:

```
main→worker  { type: "start", config, projectDir, credential: TokenHandle }
worker→main  { type: "event", event }
worker→main  { type: "approval", approvalId, call, gate, evidence }
main→worker  { type: "approval-answer", approvalId, approve, reason, author }
main→worker  { type: "token", token, expiresAt }          (refresh on request)
worker→main  { type: "token-request" }
main→worker  { type: "stop" }
worker→main  { type: "done", exitReason }
```

The worker never holds a refresh token, only short-lived access tokens handed to it by main. If the worker exits without `done`, main appends `run_aborted` to the ledger.

4.4 Credential vault

`vault.ts` stores `{ provider → encrypted blob }` in `%APPDATA%\attestqa\vault.json`, each blob produced by `safeStorage.encryptString`. If `safeStorage.isEncryptionAvailable()` is false, the app refuses to store a key and offers token-based sign-in only. Test-site logins use the same vault under `site:<origin>`, and are injected by the worker into `browser.fill` calls without ever entering a model prompt.

5. WebApplication (`@attestqa/server`)

Route	Method	Purpose
`/api/projects`	GET	Projects the user can see
`/api/runs`	POST	Start a run (approver or runner role)
`/api/runs/:id/events`	GET (SSE)	Live tail, resumable with `Last-Event-ID` = ledger `eventId`
`/api/runs/:id/report`	GET	Counted report
`/api/runs/:id/approvals/:aid`	POST	Answer; requires approver role; author from the OIDC token
`/api/runs/:id/rulings`	POST	Record a ruling
`/api/ask`	POST	Ledger query
`/api/projects/:p/coverage`	GET	Coverage grid for a work query
`/mcp`	Streamable HTTP	MCP tools `run_story`, `run_status`, `list_runs`; resources `aqa://runs/<id>`, `aqa://runs/<id>/report`

Workers run as containers with the project mounted read-write and nothing else. The server injects short-lived tokens the same way main does on the desktop. There is no MCP tool that answers an approval.

6. TestRunApplication layout

```
TestRunApplication/
  targets.yaml                  registry: every site, permission basis, rules (AQ-06)
  juice-shop/                   one Playwright project per target
    stories/*.md                stories written before any run, hashed in research/
    contracts/                  OpenAPI files, mapping documents (oracles)
    features/  pages/
    tests/web/  tests/api/  tests/security/  tests/etl/
    playwright.config.ts  package.json  ai-quality.config.yaml
    .aqa/runs/<id>/events.jsonl
  saucedemo/
  ...
```

`stories/` matters for the research: for sites without a tracker, the stories are written and hashed before the agent sees the site, so nobody can say the stories were tuned to the results.

7. Error handling

Condition	Where caught	Ledger	UI
Model HTTP 401/403	provider	`inference` with `error`	"Sign in again"; run paused

Condition	Where caught	Ledger	UI
Token expired mid-run	worker token-request	none	silent refresh; if it fails, as above
Tool throws	adapter	`observation ok:false`	shown on the failure card
Gate refusal	gate / rules	`policy` refuse + reason	violet refusal card
Approval timeout	worker	`approval_resolved {approve:false, reason:"timeout"}`	run stops cleanly
Worker crash	main	`run_aborted`	banner with "open ledger"
Disk full	ledger writer	best effort `run_aborted`	banner naming the folder

8. Logging

Application logs (not the ledger) go to `%APPDATA%\attestqa\logs\`, rotate at 5 MB, keep 5 files, and pass through the same scrubber as model requests. No telemetry is sent anywhere unless a later release adds an opt-in, which would need its own sign-off.